

A Multi-Agent Course Recommendation System

Bhatta Mridula Chandrika
bmch@kth.se
Royal Institute of Technology (KTH)
Stockholm, Sweden

Abstract

Students choosing courses often have vague preferences and several constraints at once, such as topic, study period, credits and examination form. In earlier work, CourseGPT, a system that turned natural language queries into database filters with a single language model call, helped students find suitable courses 2.5 times faster than the KTH course catalogue, but it could not answer follow-up questions, explain its recommendations or scale beyond a manually collected dataset. This paper presents a multi-agent redesign. An offline pipeline scrapes and extracts 379 courses from KTH’s public course pages with a self-correcting extraction agent. At query time, a router directs each message to a retrieval agent that writes, tests and repairs its own SQL on a sandboxed, read-only database, or to an advisor that answers follow-up questions and is fact-checked by a model from a different family before its answer is shown. We evaluate the system with 22 scripted conversation turns, checking hard constraints with rules and grading query quality and advice with two LLM judges from different model families. All searches met the stated constraints, routing was correct in every turn, and the judges agreed closely; where they disagreed, the judge from the same model family as the agents was consistently more lenient.

Keywords

course recommendation, multi-agent systems, large language models, text-to-SQL, LLM-as-a-judge, conversational recommender systems

1 Introduction

Universities offer hundreds of courses each semester, and students must choose among them while satisfying several constraints at once: topic, credits, study period, examination form and programme requirements. Course catalogues support this through keyword search and filters (Figure 1), which work well when students know the right course names and codes, but poorly when their preferences are vague, such as “an easy course about sustainability without a written exam”.

In earlier work we built CourseGPT [1], in which a large language model (LLM) translated such requests into a JSON object of filters, from which a rule-based builder generated a parameterized SQL query. In a user study with nine students, participants found courses that fulfilled the task requirements significantly faster with CourseGPT than with the KTH course catalogue (mean 70 s versus 177 s, $p < 0.001$), although usability and mental workload did not differ significantly. The study also exposed limitations. Each query was handled in isolation, so students could not refine results or ask about them. The system gave no reasons for its recommendations. Its dataset consisted of 57 manually collected courses, and the fixed filter schema could only express constraints that its rules anticipated.

The screenshot shows a web interface for searching KTH courses. At the top, there is a search bar labeled 'Search by course name or course code' with a 'Search course' button. Below the search bar, there is a link 'Filter your search choices'. The interface is divided into two main sections: 'Course Start 2026' and 'Course Start 2027'. Each section has a list of checkboxes for different periods: Spring 2026 period 3, Spring 2026 period 4, 2026 summer, Autumn 2026 period 1, and Autumn 2026 period 2. There is also an 'Educational level' section with checkboxes for Pre-university level, First cycle, Second cycle, and Third cycle. A 'School, department, etc' section has a dropdown menu set to 'All schools'. On the right side, there is a 'Course Start 2027' section with checkboxes for Spring 2027 period 3, Spring 2027 period 4, and 2027 summer. Below this, there is an 'Other options' section with checkboxes for 'Courses taught in English', 'Courses that deal with environment, environmental technology or sustainable development', and 'Dormant/Terminated course'.

Figure 1: The KTH course search, which relies on course names, codes and filters.

This paper addresses these limitations by redesigning the system as a set of cooperating agents. Our contributions are:

- an offline data pipeline that scrapes course pages and extracts structured records with an LLM, validated and corrected against rules in code;
- a conversational architecture with a router, a retrieval agent that writes and repairs SQL inside a sandboxed database, an advisor for follow-up questions and a verifier that fact-checks its answers;
- an evaluation method that combines rule-based checks with two LLM judges from different model families, and an analysis of where the judges disagree.

2 Related Work

LLM-based course recommendation. Students expect course recommenders to support subject-based retrieval rather than searching by course titles [2]. Van Deventer et al. [7] generate an ideal course description from a student’s query and retrieve courses by embedding similarity. Hill et al. [3] recommend academic programmes from topics mined with BERTopic. Both focus on semantic similarity of course text and do not combine it with hard constraints such as periods or examination form, or with course statistics and student feedback.

Conversational and LLM-based recommenders. Conversational recommenders elicit preferences through dialogue; Valtolina et

Table 1: Agents and the guarantees enforced in code.

Agent	Role	Enforced in code
Extraction	Course page to structured record	Rule checks; errors sent back, two retries
Router	Chooses search, refine, advise or reply	Search when nothing is shown yet
Retrieval	Writes, tests and repairs SQL	Read-only sandbox; system runs final query
Advisor	Answers questions, highlights courses with reasons	May only highlight shown courses
Verifier	Fact-checks the advisor’s answer	One revision at most; never blocks

al. [6] use one to help teachers find learning objects. Surveys of LLM-based recommendation highlight the ability of LLMs to interpret natural language, combine heterogeneous information and explain recommendations [8], but also risks such as unfair or inconsistent recommendations [10].

Agents and self-correction. LLM agents interleave reasoning with actions such as tool calls and use the observations to decide the next step [9]. Self-Refine [4] shows that feedback on a draft can be used to revise it. We apply these patterns to query writing, where the database provides the feedback, and to advice, where a separate verifier does.

LLMs as judges. Strong LLMs can grade open-ended output with agreement comparable to human raters [11], but they tend to prefer text generated by themselves [5]. We therefore use judges from two model families and report where they disagree.

3 Design Goals

Based on the limitations of CourseGPT, we set five goals for the new design:

- (1) **Conversation:** students can narrow, re-sort or ask about the results instead of starting over.
- (2) **Explanations:** recommendations come with reasons grounded in course data and student reviews.
- (3) **Coverage:** course data is collected automatically from the university’s own pages rather than entered by hand.
- (4) **Safety and completeness:** model-generated queries cannot modify data, and no matching course is silently dropped.
- (5) **Measurability:** every step can be evaluated, with facts checked by code and judgement calls graded by independent models.

4 System Design

Figure 2 shows the architecture. It consists of an offline data pipeline, which builds the course database, and an online conversation pipeline, which only reads from it. Table 1 summarises the agents.

4.1 Data Pipeline

The lists of second-cycle courses for a school are obtained from the endpoint used by KTH’s own course search page, and each course page is then downloaded. The scraper respects `robots.txt`, waits

between requests, backs off when the site responds slowly and caches every list and page, so an interrupted run resumes where it stopped. Because scraping is done offline, the conversation pipeline never depends on the website being available.

An extraction agent (Gemini 2.5 Flash with a fixed output schema) turns each page into a structured record: credits, study periods, examination modules, grading scale, prerequisites, learning outcomes and programmes. Rules in code then validate the record: the credits of the examination modules must add up to the course total, every study period must appear on the page, and codes and grading scales must be well-formed. Violations are returned to the model, which gets up to two attempts to correct them. Derived fields that later agents filter on, such as whether a course has a written exam (oral exams excluded), are computed in code from the structured examination modules.

The resulting database contains 379 second-cycle courses: 363 from the School of Electrical Engineering and Computer Science (EECS), 11 from the School of Engineering Sciences and 5 from other schools. Pass rates, difficulty ratings and reviews from a student survey exist for 57 courses. For the remaining 322 courses we generated placeholder ratings and reviews to demonstrate the interface; they are stored with a separate source label and marked as demo data wherever they are shown.

4.2 Router

Each message is classified by the router into one of four routes: a new *search*, a *refinement* of the results currently shown (“only the ones in P2”, “sort them by rating”), an *advice* question about them (“which of these is easiest?”), or a direct *reply* for greetings, off-topic messages and requests too vague to search. The router also rewrites the message into a self-contained request, since later agents do not see the conversation.

4.3 Retrieval Agent

In CourseGPT, the model only produced filters and a rule-based builder wrote the SQL. This was safe but limited to the fields and phrasings the rules anticipated. The retrieval agent instead writes SQL itself, in the loop shown in Figure 3. It runs a test query, observes the row count and a sample of rows or the database error, and revises the query, for example by broadening topic terms when nothing matches while keeping every condition the student stated as required. When it submits its final query, the system executes it and returns every matching course, so no model decides which results to drop.

Safety is enforced by the database layer rather than the prompt. Queries run on a read-only connection, SQLite’s authorizer only permits reading the two course tables, a single SELECT statement is allowed and queries are stopped after five seconds. Nine test queries that attempted to delete or update data, read other tables, attach databases or run indefinitely were all rejected. The system also rejects LIKE terms of three characters or fewer on free-text columns, a rule added after the evaluation revealed that such terms match inside unrelated words (Section 6).

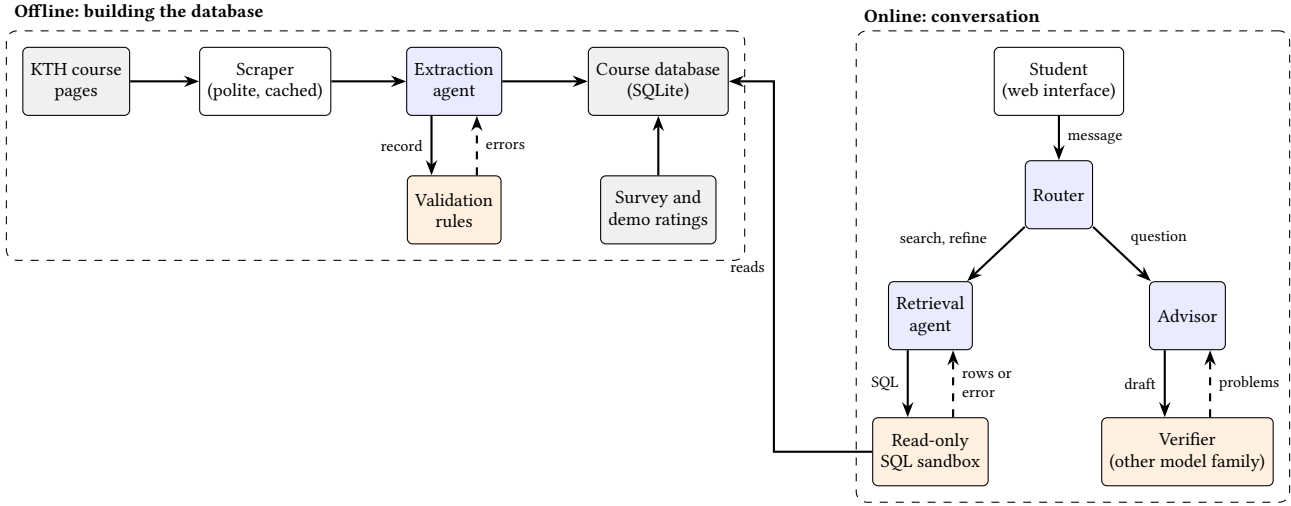


Figure 2: Architecture. The offline pipeline builds the course database; the online agents only read from it. Dashed arrows are feedback used for self-correction.

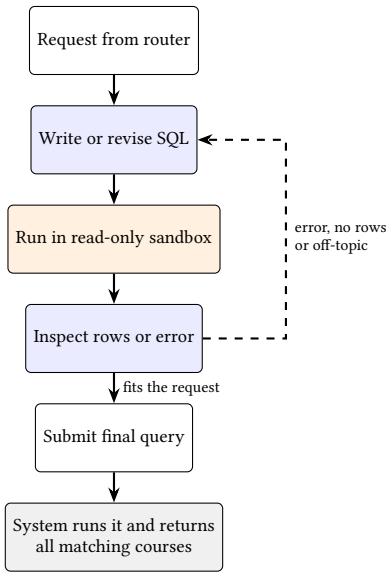


Figure 3: The retrieval agent’s loop (at most eight model calls).

4.4 Advisor and Verifier

The advisor answers questions about the courses currently shown. It receives their full records, including student reviews, with every generated value labelled as demo data, and returns an answer together with the courses it recommends and a reason for each. Only courses among the current results can be highlighted.

Before an answer is shown, a verifier based on a model from a different family (OpenAI GPT-6 Luna) checks it against the same data for three kinds of problems: unsupported or contradicted facts, demo data used without a label, and highlighted courses whose reason is not supported. If it finds any, the advisor revises its answer

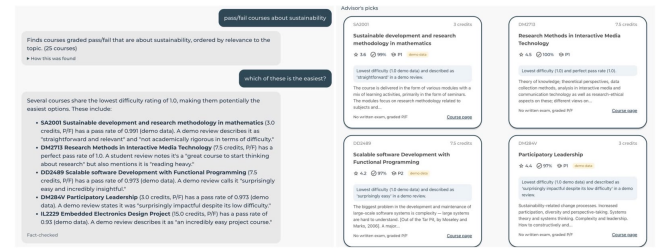


Figure 4: “How this was found”: the query the retrieval agent tested and then submitted as its final query.

once, with the problems as feedback. A failed check is recorded but never blocks the answer.

4.5 Interface

The web interface (Figure 4) presents the conversation above the results. Courses are shown as cards with credits, rating, pass rate, study periods, examination form and a link to the official course page, and a badge marks generated ratings. After an advice question, the recommended courses are shown first with their reasons. Each search can be expanded to show the queries the retrieval agent tried, including any failed attempts and corrections (Figure 5), and each answer shows whether it passed the fact check or was revised.

4.6 Implementation

The backend is written in Python with FastAPI and SQLite; the frontend uses React. All agents run on Gemini 2.5 Flash through the Google Gen AI SDK; the verifier and one of the evaluation judges use OpenAI models. Conversations are kept in memory per session. A search takes about four seconds on average, most of it spent in model calls.

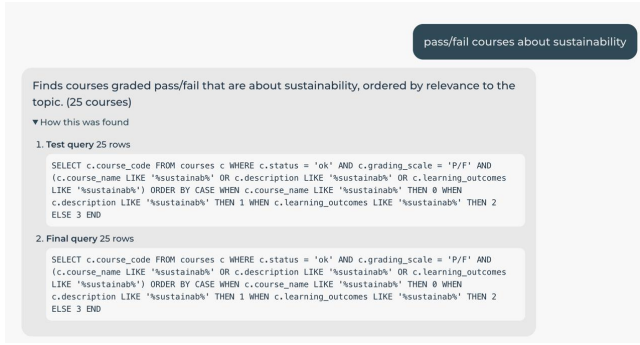


Figure 5: “How this was found”: the query the retrieval agent tested and then submitted as its final query.

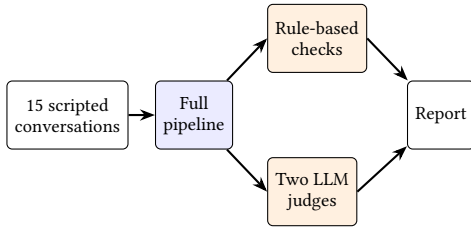


Figure 6: Evaluation setup. Every turn is checked by rules where there is a correct answer and graded by two judges where judgement is needed.

5 Evaluation Method

We evaluated the system with 15 scripted conversations comprising 22 turns: 15 turns that search for or refine courses (14 of them with constraint checks), 4 advice questions and 3 messages that require a direct reply. The conversations cover topics such as machine learning, sustainability, robotics and space physics, and constraints on study periods, grading, examination form, credits, language and programme. Figure 6 shows the setup.

Rule-based checks. Everything with a single correct answer is checked in code: whether the router chose the expected route, whether every returned course satisfies each stated constraint, and whether predefined key courses are among the results. Before use, every expected result was verified against the database, so that failures reflect the system rather than the test.

LLM judges. Aspects that require judgement are graded with written rubrics: how faithfully the SQL captures the request (1–5), the relevance of each of the top ten results, and whether the search summary is honest; and for advice, whether it is grounded in the course data, answers the question, is complete and clear (1–5 each), and labels demo data. Because judges tend to favour their own model family [5], the same outputs were graded by GPT-6 Sol (OpenAI) and Gemini 3.1 Pro (Google), while the agents run on Gemini. We report the scores of each judge and their agreement, and inspected every step where they clearly disagreed.

Verifier ablation. To measure the verifier’s contribution, the evaluation was run with and without it, graded by the OpenAI judge.

Table 2: Results on 22 turns. Both judges graded the same outputs.

Checked by rules	Result	
Routing accuracy	100%	
Searches meeting all constraints	100%	
Key courses found	100%	
Graded by judges	OpenAI	Gemini
SQL faithfulness (1–5)	4.87	5.00
Relevance of top 10 results	0.90	0.98
Honest search summaries	93%	100%
Advice grounded in data (1–5)	5.00	5.00
Advice answers the question (1–5)	5.00	5.00
Advice completeness (1–5)	4.75	5.00
Advice clarity (1–5)	5.00	5.00
Demo data labelled	75%	100%

Table 3: Problems found during development and how they were solved.

Problem	Solution
Short terms such as “ML” matched inside other words	Rule in code rejects LIKE terms of ≤ 3 characters
Results trimmed as they passed between agents	System executes the final query and returns all matches
“No exam” also excluded oral exams	has_written_exam computed from exam modules
Explicit sort requests overridden by topic ranking	Explicit sort placed first in the ordering
Extractor invented study periods (3 of 378 courses)	Periods must appear on the page, else retry
Generated reviews quoted without a label	Label attached to every generated value
Website unavailable or rate limiting	Offline, cached, resumable scraping with backoff

6 Results

Table 2 shows the results. The router chose the correct route in all 22 turns, every course returned by every search met the stated constraints, and all key courses were found. The judges rated the queries as faithful to the requests and the advice as grounded and clear.

Findings during development. These results were not reached in the first run, where only 71% of searches met all constraints. Table 3 lists the main problems the evaluation and manual testing revealed. The most severe was that the retrieval agent searched for abbreviations such as “ML” and “AI” as substrings, which also match words like “HTML” and “detail”; the search for machine learning courses returned 331 of 379 courses. After adding a rule that rejects such short terms, the same request returned 37 courses, all matching machine learning or artificial intelligence terms. The agents are not deterministic: in one later run a single search included three loosely related courses, so constraint compliance varied between 86% and 100% across the final runs.

Table 4: Agreement between the two judges on the same outputs.

Metric	Agreement
SQL faithfulness	87% identical (mean gap 0.13)
Relevance verdicts per course	90%
Honest summary	93%
Groundedness, answers question, clarity	100% identical
Completeness	75% identical (mean gap 0.25)
Demo data labelled	75%

Judge agreement. The two judges gave identical scores for 87% of the SQL grades and for all groundedness, answer and clarity grades, and agreed on 90% of the individual relevance verdicts (Table 4). They clearly disagreed in five steps. On manual inspection, the OpenAI judge’s verdict was the more defensible in all five: the Gemini judge rated courses as relevant that only mention a topic in passing, such as research methods courses in a search for sustainability courses, and accepted an answer that quoted a generated review without labelling it. This is consistent with the self-preference bias reported by Panickssery et al. [5], although five cases are too few to establish it.

Verifier. With the verifier enabled, all four advice answers passed the check without revision, and the OpenAI judge’s scores were unchanged within noise (groundedness 4.75 with and without the verifier). On this test set the verifier therefore had no measurable effect, largely because the labelling problem it was designed to catch had already been solved by attaching labels to the data. The average time per turn was almost unchanged (4.29 s without and 4.31 s with the verifier), although only four of the 22 turns involve it.

Extraction quality. An automated comparison with the page text found correct credits and grading scales for all 84 courses checked during development. Before the period check was added, the extraction agent invented study periods for 3 of 378 courses (0.8%).

7 Discussion

Dividing work between models and code. The most effective changes moved guarantees from prompts into code: the database sandbox, executing the final query instead of passing results between agents, computing examination form from structured data, and rejecting short search terms. The models are used where judgement is needed: interpreting the request, choosing search terms, correcting failed queries and explaining trade-offs. The first CourseGPT achieved safety by preventing the model from writing SQL; the multi-agent design achieves it by restricting what the SQL can do, which gives the model more freedom while keeping the same guarantees.

Evaluating with LLM judges. Rule-based checks found the most serious bug, and the judges found problems that rules cannot express, such as loosely related results and misleading summaries. Using two judges from different families was informative: a single Gemini judge would have reported near-perfect scores and missed

the problems the OpenAI judge identified. The judges have, however, only been validated by manual inspection of the cases where they disagreed.

Limitations. The system has not been evaluated with users, so its effect on search time, usability and mental workload compared to CourseGPT and the KTH catalogue is unknown. The test set is small, with only four advice questions, which was insufficient to measure the verifier’s benefit. Keyword search in SQL cannot always distinguish a course about a topic from one that only mentions it, which accounts for most remaining relevance errors. The data covers mainly one school, most ratings and all reviews beyond the original 57 courses are placeholders, and the course lists come from an undocumented endpoint that may change.

Future work. The next steps are a user study comparing the multi-agent system with CourseGPT and the KTH catalogue using the same tasks and measures as before; combining SQL filters with semantic similarity [7] to improve relevance; a larger and harder set of advice questions to evaluate the verifier; completing the data for all schools and collecting real student reviews; and extending the pipeline to other universities.

8 Conclusion

We redesigned a course recommendation system as a set of cooperating agents: an extraction agent that builds the course database from public course pages, a router, a retrieval agent that writes and repairs its own SQL in a sandboxed database, and an advisor whose answers are fact-checked by a model from another family. The design supports conversation, explains its recommendations and scales beyond a manually collected dataset. An evaluation combining rule-based checks with two LLM judges showed that it meets the stated constraints in every search and that a judge from the same model family as the agents is more lenient, underlining the value of independent evaluators. A user study is needed to establish whether these technical improvements also make course selection easier for students.

References

- [1] Elias Bäckström, Bhatta Mridula Chandrika, and Jiasui Cao. 2026. CourseGPT: Creating and Evaluating a Generative AI-Based Course Recommendation System. (2026). Royal Institute of Technology (KTH), Stockholm.
- [2] Seungeon Cha, Martin Loeser, and Kyoungwon Seo. 2024. The Impact of AI-Based Course-Recommendation System on Students’ Course-Selection Decision-Making Process. *Applied Sciences* 14, 9 (2024), 3672.
- [3] Alessandro Hill, Kalen Goo, and Puneet Agarwal. 2025. Recommending the right academic programs: an interest mining approach using BERTopic. *Data Mining and Knowledge Discovery* 39, 3 (2025), 20.
- [4] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Stefano Valtolina, Ricardo Anibal Matamoros, and Francesco Epifania. 2024. Design of a conversational recommender system in education. *User Modeling and User-Adapted Interaction* 34, 5 (2024), 1613–1641.
- [7] Hugh Van Deventer, Mark Mills, and August Evrard. 2024. From Interests to Insights: An LLM Approach to Course Recommendations Using Natural Language Queries. arXiv:2412.19312 [cs.LG] <https://arxiv.org/abs/2412.19312>
- [8] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. 2024.

- A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.
- [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- [10] Jizhi Zhang, Keqin Bao, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Is ChatGPT Fair for Recommendation? Evaluating Fairness in Large Language Model Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. ACM, 993–999.
- [11] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.